A Lesson in Lambda Calculus

An interactive website to help students learn and practice lambda calculus, built for CSE 340 at ASU, and open to anyone.

Student: Nolan Gabaldon

Thesis Director: Professor Rida Bazzi

Second Committee Member: Professor Justin Selgrad

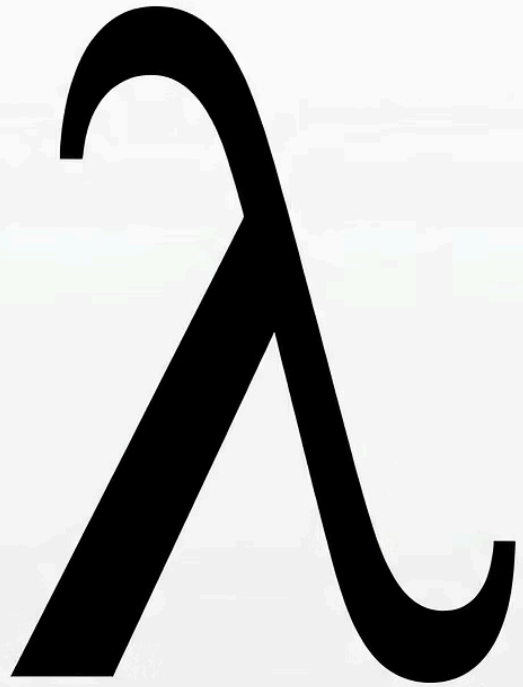


Table of Contents

BACKGROUND

Why Lambda Calculus Matters

- Lambda calculus teaches the **functional paradigm**, which helps reduce bugs from unexpected side effects and models certain problems more naturally.
- Useful beyond purely functional languages, ideas like immutability and avoiding hidden inputs/outputs apply to any language with functions.
- Covered in CSE 340 at ASU as part of a broad CS survey.

BACKGROUND

The Problem: Students Struggle

Two main reasons students have difficulty with lambda calculus:

- Lack of interactive ways to practice
- Lambda calculus mechanics are very different from anything students have seen before

Observed both as a student in CSE 340 and as an Instructional Assistant.

Existing Resources

- Example problems
- Step-by-step interpreters
- Game-like simulators

The Gap

No single tool combines all of these, and existing tools require students to already understand the topic well enough to work with limited feedback.

BACKGROUND

Thesis Goal

The goal of this thesis is to create a website that allows students to learn and practice various aspects of the lambda calculus to gain a deeper understanding of both the lambda calculus and the functional paradigm.

The claim behind this thesis is that this website is expected to improve user understanding of the topics it covers.

Novel Features

- Procedurally generated questions
- Checks user answers
- Purpose-built UI for entering answers

Scope

- Teaches basic definitions through text
- Covers a limited set of topics
- Not a replacement for existing courses
- Does not answer conceptual questions

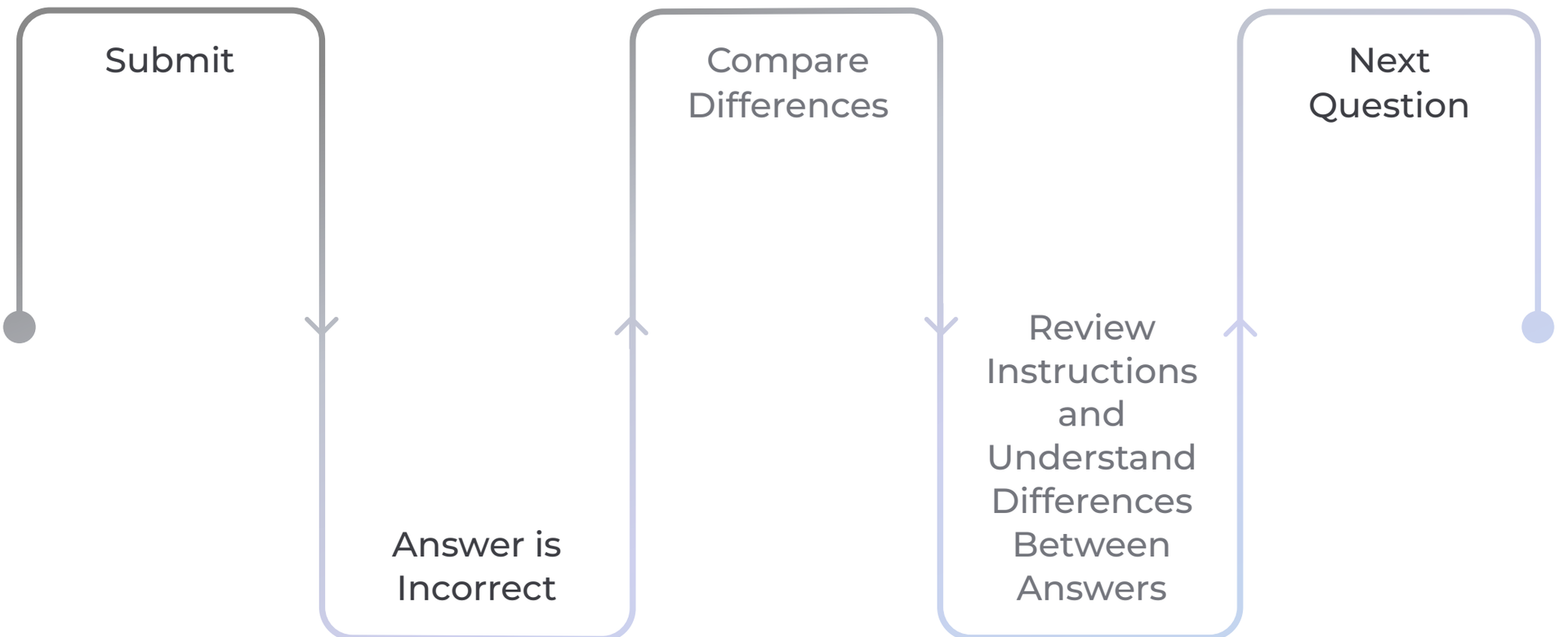
Evaluation

- Before/after user surveys
- CSE 340 exam performance data

LESSON STRUCTURE

Common Lesson Components

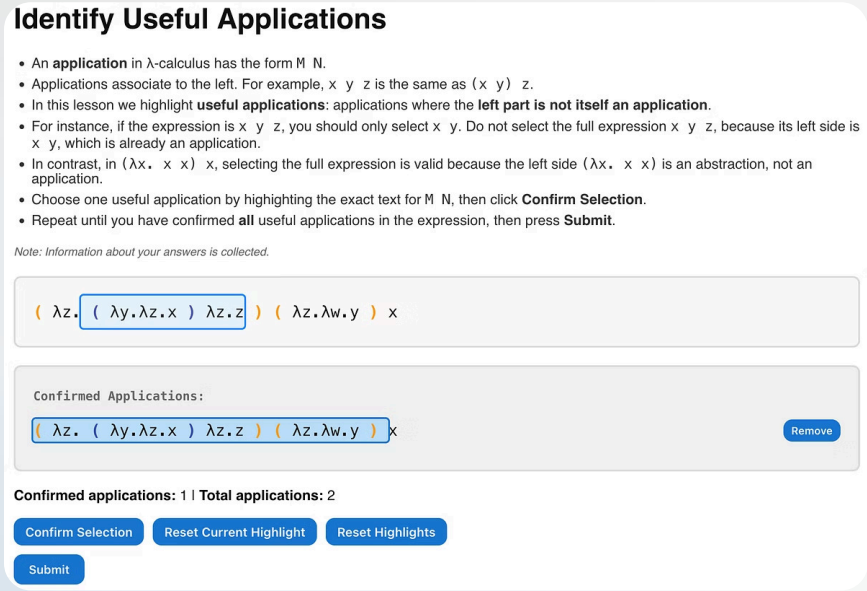
- The goal of each lesson is to build up the user's abilities toward performing a full beta reduction with renaming if needed.
- Submit a correct answer → move to the next question.
- Wrong answer → option to try again, move on, or see the correct answer.
- This creates a **rapid reinforcement loop**: submit → answer is incorrect → compare differences between user answer and given answer → review instructions → understand why they are different → next question.
- Parentheses are **color-coordinated** by nesting depth, making it easy to identify which parentheses belong together.



LESSON 1

Identify Useful Applications

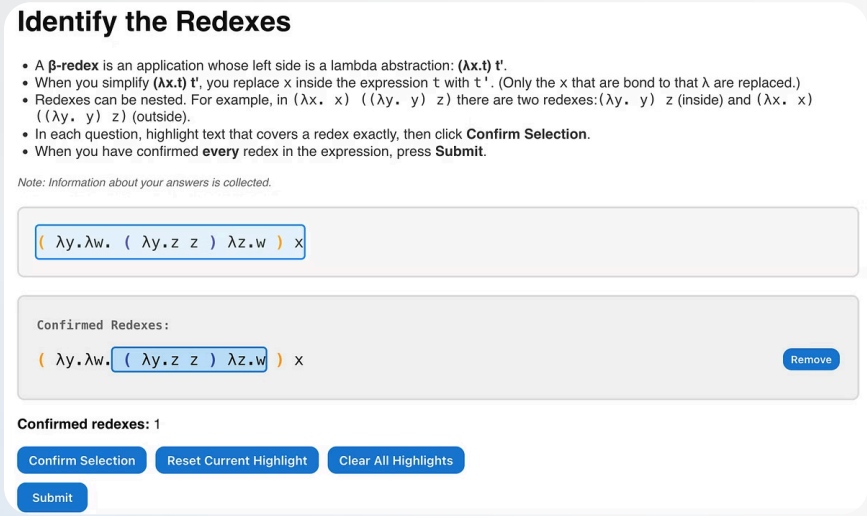
- An **application** has the form *M N*. A **useful application** is one whose right side is *not* itself an application.
- The user drags to highlight each useful application, then clicks **Confirm Selection**. Repeat for all, then **Submit**.
- Highlighting lets the user communicate where an application starts and ends with minimal friction.
- Already-confirmed applications are shown so the user knows what has already been entered.
- Questions are filtered to avoid excessive length or too many useful applications.



LESSON 2

Identify the Redexes

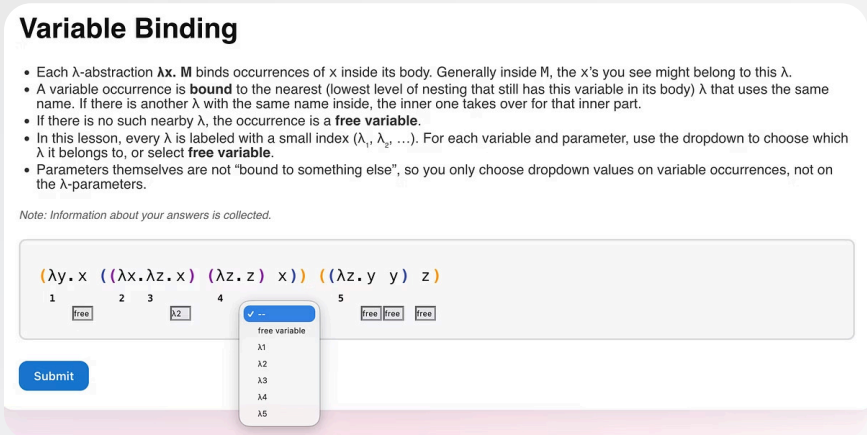
- A **redex** is a useful application whose left side is a lambda abstraction: *(λx. t) t'*.
- The user drags to highlight each redex, then clicks **Confirm Selection**. Repeat for all, then **Submit**.
- Highlighting lets the user communicate where a redex starts and ends with minimal friction.
- Already-confirmed redexes are shown so the user knows what has already been entered.
- Builds on Lesson 1: all redexes are useful applications, but the left side of the application must be an abstraction.
- Questions are filtered to avoid excessive length or too many redexes.



LESSON 3

Variable Binding

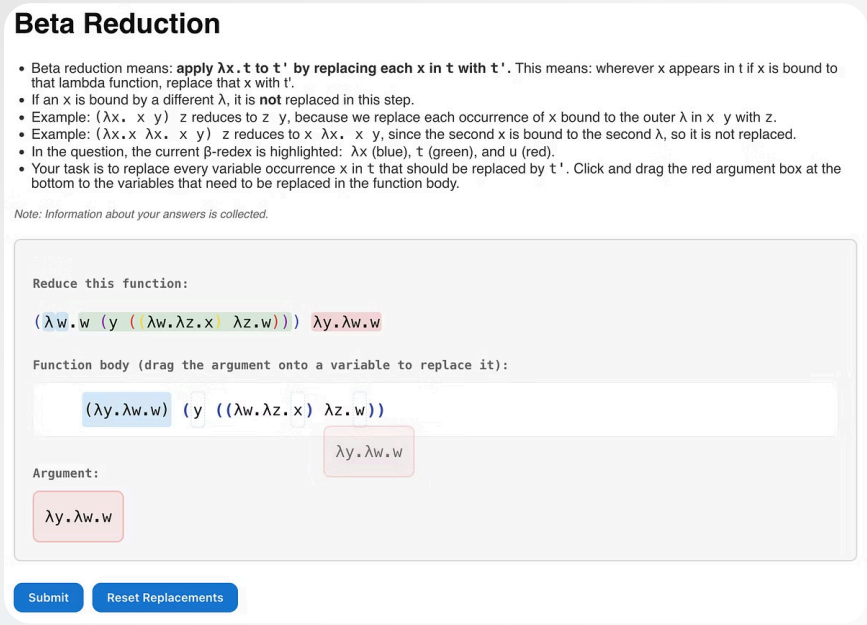
- Each variable occurrence has a **dropdown menu** to select which lambda abstraction it is bound to, or mark it as a **free variable**.
- Selecting a binding takes just two clicks: one to open the dropdown, one to choose the abstraction.
- **Design choice:** always-visible menus would reduce clicks, but would clutter the page and make it harder to parse which abstraction each variable belongs to.
- Questions are generated to ensure a good mix of both **bound** and **free** variables.



LESSON 4

Beta Reduction

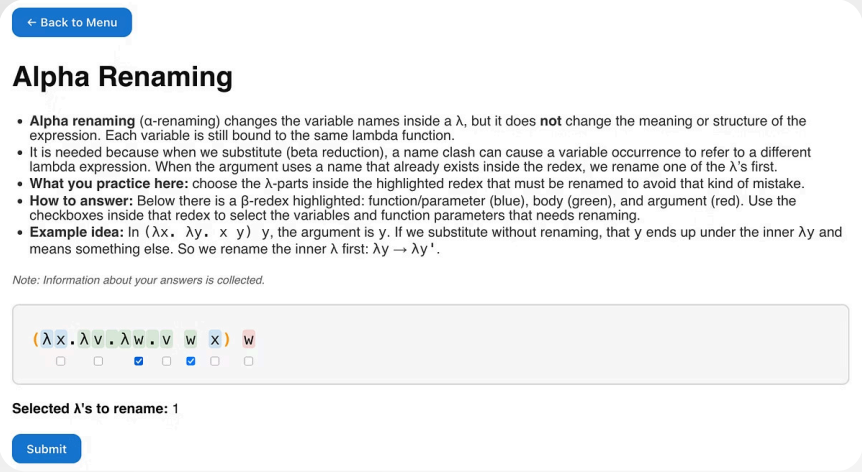
- The user **drags the argument** onto each variable that gets replaced in the beta reduction of the redex shown on the first line.
- Seeing the expression change with each drag is designed to feel more meaningful than simply marking variables for replacement.
- This interaction reinforces the substitution process and makes the lesson easier to remember.
- Requires knowledge of variable binding from Lesson 3, only variables **bound to the outer redex** are replaced.
- Questions are generated as lambda abstractions with a maximum expression length to keep problems manageable.



LESSON 5

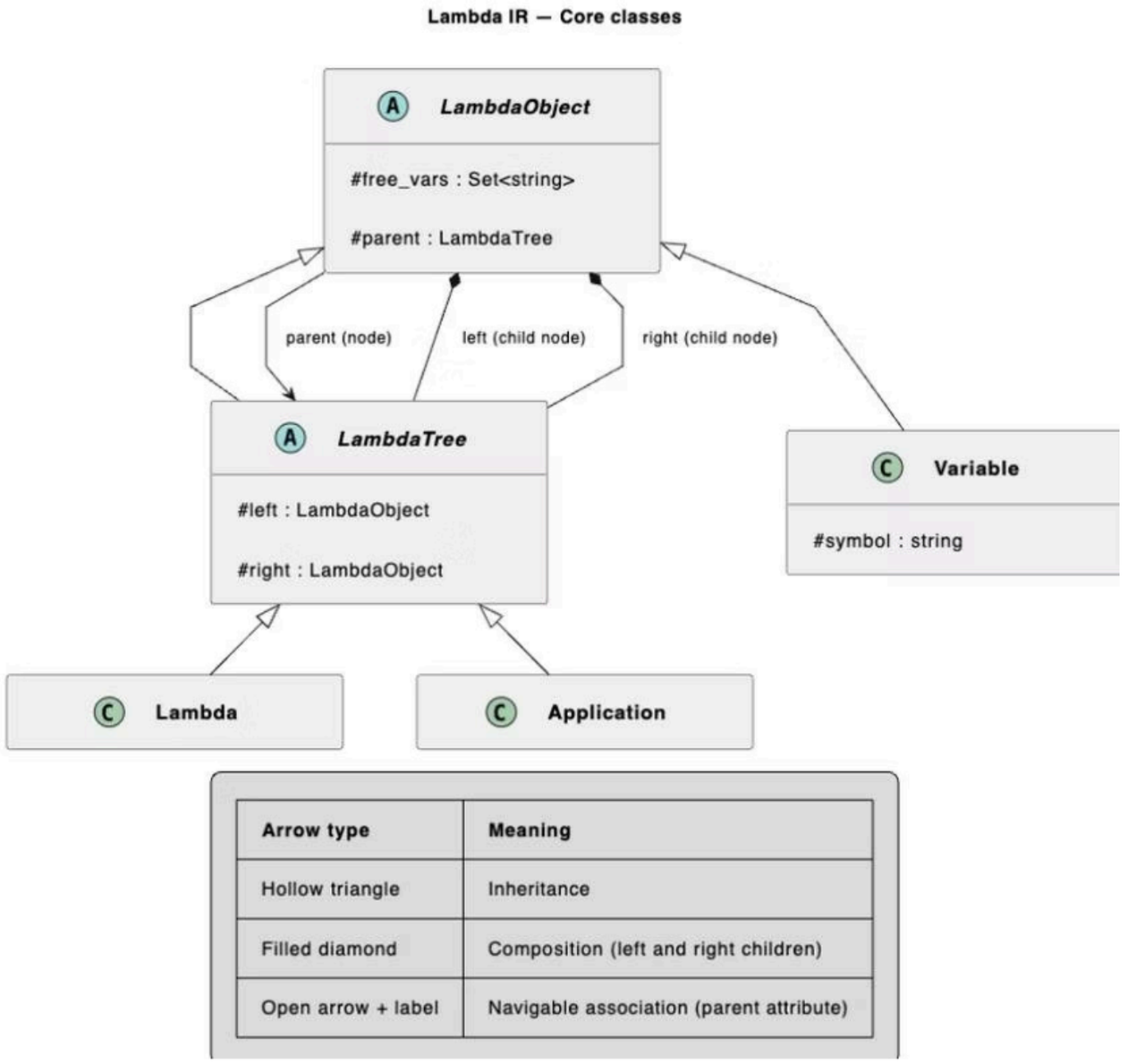
Alpha Renaming

- The user clicks a **checkbox** next to each variable that should be renamed before beta reduction.
- Checkboxes of variables bound to the same abstraction, including the abstraction's parameter, are **linked together**, reinforcing that all variables sharing a name must be renamed consistently.
- Checkboxes provide a clear, low-friction way to communicate which variables need renaming, while keeping the user's current answer visible at a glance.
- Requires knowledge of both **variable binding** and **beta reduction** to determine whether a name clash will occur during substitution.
- Questions are generated to ensure renaming is usually (but not always) required, each abstraction within the body has a variable bound to it, and the overall expression length stays manageable.



TECHNICAL DESCRIPTION

Lambda IR — Core Classes



- **LambdaObject**: abstract base for all IR nodes; tracks free variables and parent.
- **LambdaTree**: abstract; parent to Lambda and Application; holds left and right children.
- **Lambda**: concrete; represents a lambda abstraction with a parameter and body.
- **Application**: concrete; represents application of two LambdaObjects.
- **Variable**: concrete; inherits directly from LambdaObject; a single variable with no children.

TECHNICAL DESCRIPTION

Key IR Methods: `replace` and `alpha_rename`

`replace` (LambdaObject)

- Replaces all locally free instances of a variable with a given expression.
- `Application` and `Lambda` replace matching children, then recurse, but only if the variable is in their locally free variables.
- `Lambda` ignores calls targeting its own parameter (it is bound, not free).
- `Variable` ignores all `replace` calls, it cannot replace itself and has no children.
- If `Lambda`'s parameter is locally free in the incoming expression, it calls `alpha_rename` on itself first to avoid variable capture.

`alpha_rename` (LambdaObject)

- Prepares an object for a `replace` call by renaming lambda parameters and their bound variables to prevent variable capture.
- `Application` and `Lambda` recurse only if the variable being replaced is locally free in that subtree.
- `Lambda` ignores calls where the variable matches its own parameter.
- `Variable` ignores all `alpha_rename` calls.

TECHNICAL DESCRIPTION

Key IR Methods: `call`, `reduce`, and `eq`

`call` (Lambda)

Takes a replacement object, calls `replace` on the body, and returns the result. After that, this Lambda object is invalid and should be discarded.

`reduce` (Application)

If the left side is a Lambda, calls `call` with the right side. Replaces this Application with the result in the parent. This performs a full beta reduction.

`eq` (LambdaObject)

Recursively checks if two expressions are equivalent, ignoring renaming. Free variables must share the same name; bound variables must bind to the same structural position. Correctly identifies $(\lambda x.\ x) \equiv (\lambda y.\ y)$.

TECHNICAL DESCRIPTION

Key IR Methods: object_range, redexes, and norm_ord_redex

object_range (LambdaObject)

Returns the character indexes (excluding spaces) of each LambdaObject in the expression tree. For example, z λx. y returns ranges for z λx. y, λx. y, and y. Ranges are updated with the starting offset after recursive calls return, emulating each class's toString logic.

redexes (LambdaObject)

Recursively returns a list of all Applications whose left side is a Lambda (i.e., all redexes).

norm_ord_redex (LambdaObject)

Recursively finds and returns the left-most redex not contained within any other redex (normal-order reduction).

QUESTION GENERATION

Random Expression Generation

`random_lambda`

Takes a variable name list and a max depth. Randomly returns an `Application`, `Lambda`, or `Variable` with equal probability; returns a `Variable` at depth 0. `Application`: left side keeps max depth; right side decrements by 1. `Lambda`: picks a random variable, recurses with decremented depth. `Variable`: picks a random variable from the list. Depth decrements when entering a `Lambda` or the right side of an `Application` (not the left), producing more nested parentheses.

`random_with_unique_lambdas`

Like `random_lambda`, but: no `Lambda` may share a parameter with an ancestor `Lambda`, and every `Lambda` must have at least one variable bound to it.

Hidden Difficulty

Each lesson tracks a hidden difficulty based on correct answers submitted: Easy (≤1), Medium (2–3), Hard (4+). Primarily affects question filtering: simpler questions appear early, and harder ones appear as the user progresses.

QUESTION GENERATION

Lesson 1 & 2: Question Parameters

Identify Useful Applications

Generated via `random_lambda`, filtered by min string length (excluding spaces) and number of useful `Applications`.

- Min useful Applications: 1 for all difficulties.
- Easy: max depth 3, min length 3, max 4 useful apps.
- Medium: max depth 3, min length 4, max 6 useful apps.
- Hard: max depth 4, min length 5, max 8 useful apps.

Redex Highlighting

Generated via `random_lambda`, filtered by number of redexes and string length (including spaces).

- Easy: max depth 3, 1–2 redexes, max length 20.
- Medium: max depth 4, 2–3 redexes, max length 30.
- Hard: max depth 4, 2–3 redexes, max length 40.

QUESTION GENERATION

Lessons 3–5: Question Parameters

Variable Binding

- All: depth 4, 25–95% bound variables
- Easy: max length 40, min 2 bound vars
- Medium: max length 50, min 3 bound vars
- Hard: max length 60, min 4 bound vars

Beta Reduction

- All: depth 4, 0–3 bound vars, arg length 3–10, no alpha renaming
- Easy: body length 7–17
- Medium: body length 12–22
- Hard: body length 17–27

Alpha Renaming

- Uses `random_with_unique_lambdas`, depth 7
- 66.7% chance of renaming; 50% short arg
- Easy: max total length 20, short args only
- Medium: max 35; Hard: max 50

ANSWER VERIFICATION

How Answers Are Checked

Useful Applications & Redexes

Use `object_ranges` to find correct index ranges. Compare user's highlighted range (excluding spaces) to expected answer.

Variable Binding

Assign a number to each lambda. Find which lambda each variable is bound to. Compare with user's dropdown selections.

Beta Reduction

Find variables bound to the reduced abstraction. Check that the user replaced exactly those variable occurrences.

Alpha Renaming

Run `alpha_rename` on the expression. Mark renamed variables. Compare with user's checked checkboxes.

RESULTS

Website Survey: Setup

- Users completed a survey **before and after** using the website.
- **15 people** completed both surveys.
- Hypothesis test: one-sided, null hypothesis H_0: *p_before* ≥ *p_after*, alternative H_1: *p_before* < *p_after*. Alpha = 0.1.
- Survey questions covered: beta reduction, redex counting, parameter identification, and need for renaming.

RESULTS

Website Survey: Results Table

Survey Question	Knowledge Level	Correct Before	Correct After	Total	p-value
reduction	all levels	15	14	15	0.845
redex count	all levels	7	11	15	0.068
parameter	all levels	2	1	15	0.729
need renaming	all levels	8	8	15	0.5
redex count	familiar	6	9	13	0.117
redex count	heard of it	0	1	1	0.079

RESULTS

Website Survey: Key Finding

Only Significant Result

At α = 0.1, the only question where we can reject the null hypothesis is **redex counting across all knowledge levels** (p = 0.068).

Some evidence the website helped users improve in this area.

Caveat

With only 15 participants, it is hard to draw strong conclusions about whether the website does an effective job at teaching users overall.

RESULTS

Exam Survey: Setup

- CSE 340 students could use the website to prepare for an exam covering lambda calculus.
- One-sided hypothesis test: $H_0: p_1 \geq p_2$, $H_1: p_1 < p_2$, does one usage group have a lower probability of being correct than another?
- Alpha = 0.1.

RESULTS

Exam Results Table

Website Usage	Correct	Incorrect	Total	% Correct
Website Used	65	44	109	59.63%
Website Not Used	44	17	61	72.13%
Not Aware of Website	7	9	16	43.75%
Didn't Answer	8	7	15	53.33%
Total	**124**	**77**	**201**	**61.69%**

RESULTS

Exam Survey: Significant p-values

Usage 1	Correct 1	Total 1	Usage 2	Correct 2	Total 2	p-value
Website Used	65	109	Website Not Used	44	61	0.052
Not Aware	7	16	Website Not Used	44	61	0.016
No Answer	7	14	Website Not Used	44	61	0.055

RESULTS

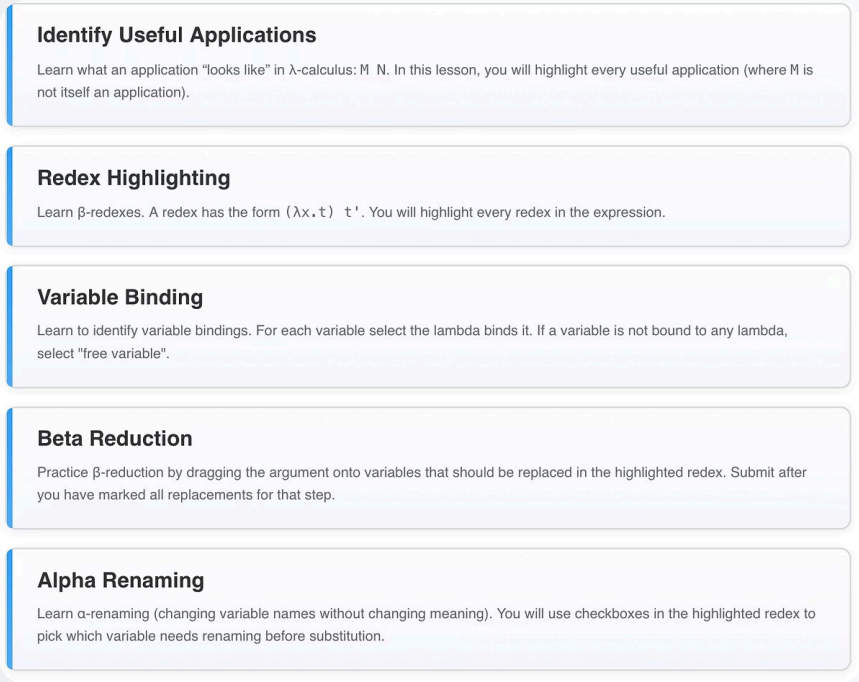
Exam Survey: Interpretation

- Students who used the website scored **lower** than those who did not, likely due to this not being a controlled experiment.
- One likely explanation: the website was made available **the day before the exam**. Students who felt prepared probably did not use it. Those who used it may have been the least prepared to begin with.
- This selection bias makes it difficult to draw conclusions about the website's effectiveness from exam data alone.

CONCLUSION

What Was Built

- A single interactive website combining guided practice, immediate feedback, and procedurally generated questions.
- Lessons build from basic pattern recognition up to beta reductions and alpha renaming.
- The IR, question generation, and answer verification systems make these interactions consistent and scalable.



CONCLUSION

What the Results Show

Promising

- Some evidence the website improved redex-counting skills in the survey.
- Addresses gaps left by existing practice resources.
- Can help students build fluency with core lambda calculus concepts.

Limitations

- Small sample sizes make broad conclusions difficult.
- Exam data is likely confounded by selection bias.
- Mixed overall evidence. Hard to say definitively whether the website improves learning.

CONCLUSION

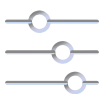
Future Work

More Lessons

Cover more advanced lambda calculus topics and add a lesson focused on terms and definitions.

Better Experiments

Improve experimental controls to get cleaner data on learning outcomes.



Fine-Tune Lessons

Further refine existing lessons to make them as effective as possible.

GITHUB & WEBSITE

Links

The website and source code are publicly available:

Website

lambda-calculus-interactive-lesson.vercel.app

GitHub

github.com/Dark-Hydra8/lambda-calculus-interactive-lesson

Works Cited: *Lambdulus* (lambdulus.github.io); Olio, JT — *Whiteboard Problems in Pure Lambda Calculus*; Potpara & Victor — *Alligator Eggs*; CS 320 Practice Problems 06 (cs.bu.edu).